

Raport SQL Profiler (EF Core)

1. Zapytanie LINQ (VisitService.GetDetailsAsync)

```
var visit = await _context.Visits
    .AsNoTracking()
    .Include(v => v.Patient)
    .Include(v => v.Doctor)
    .Include(v => v.Procedures)
    .Include(v => v.PrescribedMedications)
        .ThenInclude(p => p.Medication)
    .Include(v => v.ClinicalNotes)
    .FirstOrDefaultAsync(v => v.Id == id);
```

2. Przechwycony kod SQL

```
SELECT [t].[Id], [t].[AssignedDoctorId], [t].[PatientId], [t].[ScheduledDate],
       [t].[Status], [p].[Id], [p].[FirstName], [p].[LastName], [p].[Pesel],
       [u].[Id], [u].[Email], [u].[UserName],
       [m].[Id], [m].[Description], [m].[FinalCost], [m].[Name],
       [p0].[Id], [p0].[Dosage], [p0].[Quantity], [p0].[UnitPriceAtPrescription],
       [m0].[Id], [m0].[Name], [m0].[UnitPrice],
       [c].[Id], [c].[Content], [c].[Timestamp]
FROM (
    SELECT TOP(1) [v].[Id], [v].[AssignedDoctorId], [v].[PatientId],
                  [v].[ScheduledDate], [v].[Status]
    FROM [Visits] AS [v]
    WHERE [v].[Id] = @__id_0
) AS [t]
INNER JOIN [Patients] AS [p] ON [t].[PatientId] = [p].[Id]
INNER JOIN [AspNetUsers] AS [u] ON [t].[AssignedDoctorId] = [u].[Id]
LEFT JOIN [MedicalProcedures] AS [m] ON [t].[Id] = [m].[VisitId]
LEFT JOIN [PrescribedMedications] AS [p0] ON [t].[Id] = [p0].[VisitId]
LEFT JOIN [Medications] AS [m0] ON [p0].[MedicationId] = [m0].[Id]
LEFT JOIN [ClinicalNotes] AS [c] ON [t].[Id] = [c].[VisitId]
ORDER BY [t].[Id], [p].[Id], [u].[Id], [m].[Id], [p0].[Id], [m0].[Id]
```

3. Wnioski i Analiza

- Entity Framework Core optymalizuje zapytanie 'FirstOrDefaultAsync' wykonując subquery (SELECT TOP 1) przed dołączeniem reszty tabel. Zapobiega to pobieraniu niepotrzebnych wierszy głównych.
- Dane relacyjne (Procedures, Prescriptions, Notes) są dołączane za pomocą złączeń typu LEFT JOIN, ponieważ mogą one nie występować dla każdej wizyty.
- Mimo złączeń LEFT JOIN, metoda AsNoTracking() zapewnia, że obiekty nie trafiają do cache śledzenia EF, co znacząco zmniejsza zużycie pamięci (bardzo ważne w przypadku dużych odczytów).